

RELATÓRIO — AUDITORIA, ADEQUAÇÃO E MIGRAÇÃO SEGURA DO MÓDULO FINANCEIRO

Projeto: Lima's Fretes e Locações **Branch:** feature/financeiro-conta-corrente **Data:** 12/09/2026 **Status**
Git: Commit **NÃO realizado** · Push **NÃO realizado** · Deploy **NÃO realizado** · Merge na main **NÃO realizado**

Todo o trabalho está disponível apenas localmente (código na working tree + banco D1 local), para sua validação antes de qualquer commit.

1. Funcionamento anterior (antes da alteração)

Arquitetura descoberta na auditoria

O sistema separa **previsto** de **realizado**:

Livro	Tabela	O que é
Previsto	financial_entries	Contas a receber / a pagar (parcelas, vencimentos, status)
Realizado (entradas)	payments	Dinheiro que de fato entrou (pagamento na reserva, adiantamento recebido, recebimento de parcela, frete)
Realizado (saídas)	expenses	Dinheiro que de fato saiu (saída avulsa do Financeiro, conta a pagar baixada)

Fatos-chave encontrados:

- O campo `account_id` (FK para `financial_accounts`, `ON DELETE SET NULL`) **já existia** desde a migration `0004_financeiro.sql` em `payments`, `expenses` e `financial_entries` — mas **nunca era preenchido** nos lançamentos diretos.
- O **Contas a Pagar (compras)** já pedia Conta Corrente e gravava `account_id` corretamente.
- Todo o resto ignorava a conta: pagamento na tela da Reserva, adiantamento, saída avulsa do Financeiro, recebimento de parcela. O histórico desses fluxos ficou com `account_id = NULL`.
- Caução** (`deposits`) é garantia: entra/existe à parte, **não compõe** entradas do caixa; apenas a parte **retida** é contabilizada (como "Total retido por danos", fora do caixa). Não recebe vínculo com conta — e isso está **correto** e foi preservado.
- O **saldo** das Contas Correntes **não é armazenado**: é recalculado dinamicamente como `saldo = saldo inicial + Σ entradas (payments por account_id) - Σ saídas (expenses por account_id)` (função pura `saldoConta` em `src/lib/financeiro.ts`).
- O **Dashboard** e o **Financeiro** não consultam `financial_accounts` para seus indicadores — usam os livros globais. Logo, a vinculação não altera nenhum indicador (comprovado nos testes).

2. Funcionamento novo (após a alteração)

A mesma arquitetura, agora com a Conta Corrente amarrada ao caixa realizado:

1. **Todo lançamento que representa dinheiro entrando/saindo agora grava `account_id`:**
 - Pagamento na tela da Reserva (select "Conta Corrente", pré-preenchido com a conta única);
 - Adiantamento **recebido agora** (pago agora → vira linha de caixa com a conta);
 - Adiantamento **agendado** → conta é guardada no `financial_entries` e, quando **confirmado** (dinheiro entra), passa para o `payments` com a mesma conta (esse caminho já existia; foi verificado, não alterado);
 - Recebimento de parcela de frete ("Receber parcela" na tela do frete);
 - Saída avulsa no menu Financeiro;
 - Contas a Pagar (compras): fluxo intacto, como já funcionava.
 2. **Previsões em aberto não recebem conta como dinheiro:** gerar parcelas guarda a conta escolhida na `financial_entries` (para uso no momento da baixa), mas conta em aberto **não** vira saldo. Regra preservada.
 3. **Escolha de conta é opcional:** a opção "Sem conta (dinheiro fora das contas cadastradas)" permanece, para casos como dinheiro guardado fora das contas. O padrão da UI é a conta única existente.
 4. **Histórico migrado** (seção 12), sem tocar em valor/data/status/origem.
-

3. Pagamentos — fluxo completo

Onde é criado: tela da Reserva → seção "PAGAMENTOS" → server action `addPayment` (`src/app/(app)/reservas/actions.ts`).

Fluxo:

1. Formulário: Valor, Data, Forma de pagamento, Observação e (novo) **Conta Corrente**.
2. `addPayment` valida (reserva existe, valor > 0) e insere em `payments` **com `account_id`**.
3. `payments` É o livro de entradas — não existe "geração de entrada" separada: a linha do pagamento **é** a entrada.
4. Aparece no Financeiro (aba Entradas, filtrado por `paid_at`), entra no "Recebido" do Dashboard (via `paid`), reduz o saldo da reserva e o "A receber".
5. **Não altera** status da reserva nem gera lançamento duplicado.

E2E testado na UI: pagamento de R\$ 90 na LIMA-002 com conta selecionada → saldo da reserva zerou, Entradas 700 → 790, A Receber 948 → 858, linha no banco com `account_id = 1`. ✓

4. Adiantamentos — fluxo completo

Onde é criado: tela da Reserva → seção "ADIANTAMENTO" e também na criação da reserva (`ReservationForm`), via `criarAdiantamento` (`src/lib/receber.ts`).

Dois cenários (regra confirmada em código e agora implementada):

Cenário	Comportamento	Conta Corrente
Pago agora (imediato)	Vira linha de caixa imediatamente: <code>INSERT</code> em <code>payments</code>	Grava <code>account_id</code> (correção feita — antes perdia o vínculo)
Agendado (a receber)	Vira linha em <code>financial_entries</code> (previsto), não é dinheiro	Conta é guardada no entry; ao confirmar (dinheiro entra, via <code>confirmarAdiantamento</code> — fluxo já existente), o <code>payments</code> recebe a conta

- Adiantamento agendado **não** altera saldo da conta nem "Recebido" do Dashboard — correto: só o previsto.
- O saldo disponível para adiantar continua derivado de `total - pago`.

5. Caução — regra definida e justificativa

Auditoria: a caução vive na tabela `deposits` (valor, forma de recebimento, status `nao_recebida/recebida/devolvida/retida parcial/retida integral`, datas, valor retido, motivo).

Como o sistema a trata hoje: como **garantia**, fora do caixa:

- Não entra no faturamento; não entra em Entradas/Saídas; não altera saldo da conta.
- O Financeiro exibe apenas "Total retido por danos até hoje" (`SUM(retained_cents)` de `deposits` retidas), fora do caixa, com a nota explicativa na própria tela: "A caução não entra no faturamento: é devolvida ao cliente, exceto na parte retida."

Decisão (baseada no código, não presumida): NÃO vincular cauções à Conta Corrente. Justificativa: o dinheiro da caução não é receita nem despesa do caixa no modelo do sistema — é obrigação devolvível; a parte retida já tem seu próprio registro (`retained_cents`). Criar vínculo aqui **duplicaria** a representação financeira. Nenhum registro de caução histórico foi alterado.

6. Contas a receber — criação, baixa, recebimento e Conta Corrente

Criação: na tela da Reserva, seção "PARCELAMENTO" (`gerarParcelas`): divide o total em N parcelas (`dividirParcelas` — sobra de centavos na 1ª), vencimentos mensais (`vencimentos`), grava em `financial_entries` (direção `receber`), **com a Conta Corrente escolhida** (novo select).

Baixa/Recebimento: botão "Receber" na aba "A receber" do Financeiro (`receberParcela`):

- Suporta **parcial** (lança a diferença em `payments` e mantém a entry em aberto);
- Cria a entrada real em `payments` (agora **com** `account_id` herdado do formulário/padrão da conta);
- A situação da parcela é **derivada** (`situacaoParcela`): quitada / parcial / vencida / aberta — nunca armazenada, impossível de divergir do caixa.
- **Cancelamento** de entry marca `status='cancelada'` (preserva histórico, não apaga).
- Estorno: os livros de caixa aceitam lançamentos negativos (linhas de estorno no mesmo livro).

Regra aplicada: a conta é vinculada **no momento do dinheiro entrar** (baixa/recebimento), nunca na criação da conta em aberto. Conta a receber em aberto **não** é saldo. ✓ (conforme sua diretriz, item 9)

7. Contas a pagar — fluxo atual (inalterado)

- Compras (`compras`) geram parcelas em `financial_entries` (direção `pagar`) **já pedindo Conta Corrente** — comportamento anterior, correto, **não alterado**.
- Baixa via `payEntry` cria a saída real em `expenses` com a conta, atualiza o caixa e aparece no Financeiro/Dashboard (Despesas).
- Teste de regressão da suíte cobre esse fluxo (737 testes passando).

8. Conta Corrente — como os lançamentos são vinculados

- Tabela: `financial_accounts` (migration `0004`): `id`, `name`, `kind`, `initial_balance_cents`, `active`, `notes`.
- Banco local: **1 conta ativa**, `id = 1`, nome "**Conta Corrente**", saldo inicial R\$ 0,00.
- Vínculo: coluna `account_id` (FK `REFERENCES financial_accounts(id) ON DELETE SET NULL`) em `payments`, `expenses` e `financial_entries`.
- Novos lançamentos: select nos formulários (padrão = conta única); persistido em todas as actions do caixa (seção 2).
- Histórico: migration `0018` + script (seções 12–13).

9. Saldo — fórmula utilizada

Dinâmico, nunca armazenado (`saldoConta`, `src/lib/financeiro.ts`):

```
saldo da conta = saldo_inicial + Σ payments.account_id = conta - Σ expenses.account_id = conta
```

- As telas de Configurações > Contas consultam por `account_id`; criados índices `idx_pay_account` e `idx_exp_account` para essas somas.
- Como todos os lançamentos do caixa agora têm conta (histórico vinculado), o saldo da Conta Corrente reflete o extrato completo do negócio.
- Verificação no banco local: por conta = globais (ninguém fora, nada duplicado).

10. Dashboard > Financeiro do mês — fórmulas REAIS (lidas no código, `src/lib/queries.ts` + página)

Período: do dia 1 ao último dia do mês corrente (data de São Paulo). Sem filtros manuais.

Indicador	Fórmula real
Faturamento	Σ <code>total_cents</code> de reservas ativas (<code>status IN ativos</code>) com <code>event_date</code> no mês + fretes concluídos no mês. Não é caixa — é competência.

Indicador	Fórmula real
Recebido	$\Sigma \text{ payments.paid_at}$ no mês (todo pagamento real, de reserva, frete ou adiantamento confirmado).
A Receber	$\Sigma (\text{total_cents} - \Sigma \text{ payments da reserva})$ de todas as reservas ativas (saldo em aberto, independente do mês do evento).
Despesas	$\Sigma \text{ expenses.paid_at}$ no mês.
Lucro estimado	Faturamento – Despesas (competência, estimado — não é o resultado de caixa).

- Cauções: não entram em nenhum indicador de caixa; só alertas de saldo pendente.
- Adiantamentos pendentes aparecem como card próprio (`adiantamentosPendentes`).
- **A vinculação da Conta Corrente não altera nenhuma dessas fórmulas** — nenhuma delas consulta conta. Comprovado: números idênticos antes/depois da migração, exceto o efeito legítimo do pagamento novo de teste (Recebido 700 → 790).

11. Menu > Financeiro — fórmulas REAIS (`src/app/(app)/financeiro/page.tsx`)

Período selecionável (padrão: mês corrente, `de / ate` , filtro por `paid_at / due_date`).

Indicador	Fórmula real
Entradas	$\Sigma \text{ payments}$ com <code>paid_at BETWEEN de AND ate</code> (o caixa realizado do período).
Saídas	$\Sigma \text{ expenses}$ com <code>paid_at BETWEEN de AND ate</code> .
Resultado	Entradas – Saídas (caixa do período — <code>resultadoPeriodo</code>).
A Receber (total)	$\Sigma (\text{total_cents} - \Sigma \text{ payments da reserva})$ de reservas ativas — igual ao Dashboard (não filtra por período; é o saldo global em aberto).
Abas "A receber"/"A pagar"	Parcelas de <code>financial_entries</code> com situação derivada por <code>situacaoParcela</code> (quitada/parcial/vencida/aberta/cancelada) + totais.
Cauções	Apenas "Total retido por danos até hoje" (fora do caixa).

12. Comparação Dashboard × Financeiro (diferenças documentadas — NÃO corrigidas, fora do escopo)

Aspecto	Dashboard	Financeiro
Entradas	"Recebido" = payments do mês	"Entradas" = payments do período (mesma tabela/regra) ✓
Saídas	"Despesas" = expenses do mês	"Saídas" = expenses do período (mesma regra) ✓

Aspecto	Dashboard	Financeiro
Faturamento	Competência (reservas ativas + fretes concluídos do mês)	Não existe indicador de faturamento
Resultado	Não existe (tem "Lucro estimado" = competência – despesas)	Caixa (Entradas – Saídas)
A Receber	Saldo global das reservas ativas	Mesma coisa (idêntico por construção) ✓
Datas	event_date (competência) vs paid_at (caixa) — coexistem por design	paid_at / due_date

Conclusão: não há divergência de dados entre os dois — são recortes distintos por design (caixa vs competência). Os números batem quando o recorte coincide. Nada foi alterado aqui.

13. Histórico — exatamente o que recebeu vínculo

Critério (restritivo, não indiscriminado): somente os dois livros do **caixa realizado** — `payments` e `expenses` —, porque **por definição** cada linha deles é dinheiro que de fato entrou ou saiu. Ficaram **de fora**, por não representarem dinheiro:

- `financial_entries` em aberto (previsões — não são dinheiro até a baixa);
- `deposits` (cauções — garantia devolvível; seção 5).

Antes (banco local, dados de demonstração):

Tipo	Registros	Valor total	Sem conta
Payments	2	R\$ 700,00	2
Expenses	3	R\$ 315,00	0 (já vinculados pelo Contas a Pagar)

Depois:

Tipo	Registros	Valor total	Sem conta
Payments	2	R\$ 700,00	0
Expenses	3	R\$ 315,00	0

- Única diferença: o vínculo. **Nenhuma** quantia, data, reserva, cliente, forma de pagamento ou status foi alterada.
- Conferência de integridade: Σ por conta = Σ global, para os dois livros.
- **No seu banco de produção (D1 remoto) nada foi executado.** Quando você aplicar a migration lá, o mesmo script imprime o retrato ANTES/DEPOIS do seu histórico real.

14. Migrations

Migration	Conteúdo
migrations/0018_conta_corrente.sql (nova)	1) Cria a Conta Corrente somente se não existir nenhuma conta ativa (não toca em conta existente do administrador); 2) vincula histórico (payments / expenses com account_id IS NULL → conta ativa); 3) índices idx_pay_account , idx_exp_account . Aditiva, idempotente, sem DROP / DELETE / UPDATE de dados existentes além do vínculo.

- Migrations antigas: **nenhuma foi alterada**.
- Idempotência **provada por teste** (tests/conta-corrente.test.ts executa a migration duas vezes sobre o mesmo banco e confere que nada muda na segunda vez e nenhum valor muda na primeira).
- Rollback documentado no próprio scripts/vincular-conta-corrente.sql (reverte **apenas** o vínculo criado pela migration, reconhecível pelas notas da conta).

15. Testes

Automação:

- tests/conta-corrente.test.ts (novo, 11 testes): criação da conta só quando não existe ativa; vínculo do histórico; **idempotência (2ª execução = zero mudanças)**; preservação de valores/datas/status; pagamento novo grava conta; adiantamento imediato grava conta; adiantamento agendado não é caixa; parcelamento guarda conta na entry; caução fica fora.
- Suíte completa: **737/737 testes passando** (nenhuma regressão).
- Typecheck (tsc --noEmit): **limpo**.

Smoke test na UI (servidor local, dados de demonstração):

- Dashboard: números coerentes (Recebido R\$ 790,00 após o teste; Despesas R\$ 315,00).
- Financeiro: Entradas 790 / Saídas 315 / Resultado 475 / A Receber 858 — coerentes entre si e com o banco; histórico exibindo "· Conta Corrente" nas entradas e saídas.
- Tela da Reserva: 3 selects "Conta Corrente" presentes (Adiantamento, Parcelamento, Pagamentos), padrão = conta única.
- Financeiro > Saídas: select "Conta Corrente" presente no formulário.
- **E2E real:** pagamento de R\$ 90,00 na LIMA-002 com conta → saldo zerado, entrada criada com account_id = 1 (verificado no banco), Dashboard e Financeiro atualizados consistentemente, console sem erros.

16. Pendências / pontos de atenção

1. **Produção (D1 remoto) não foi tocada.** A migration 0018 precisa ser aplicada lá (npm run db:migrate) no momento que você decidir — ela é segura e re-executável, mas é uma decisão sua.

2. **Dados reais:** a auditoria de dados foi feita no banco **local de demonstração** (o local não possui seus dados reais). No produção, rode `scripts/vincular-conta-corrente.sql` uma primeira vez **antes** da migration para ver o retrato ANTES/DEPOIS do seu histórico real, sem executar nada.
3. **Receber parcela** (Financeiro > A receber): a entrada criada usa o select de conta do formulário de saídas/entradas conforme já existia no fluxo — o padrão é a conta única. Se você quiser um select explícito **no modal de recebimento da parcela**, é uma melhoria pequena e não bloqueante.
4. **Caução recebida** não gera caixa por design do sistema (seção 5). Se no seu negócio real a caução **entra** no dinheiro da conta, isso seria uma mudança de regra de negócio — não foi feita, conforme a instrução de não presumir.

17. Arquivos alterados / criados

Criados (3):

Arquivo	Motivo
<code>migrations/0018_conta_corrente.sql</code>	Estrutura incremental: garantir conta ativa, vínculo histórico idempotente, índices.
<code>scripts/vincular-conta-corrente.sql</code>	Mesma lógica executável e re-executável, com auditoria ANTES/DEPOIS e rollback documentado, para uso local e no produção.
<code>tests/conta-corrente.test.ts</code>	Provas automatizadas: idempotência, preservação de dados, persistência da conta nos novos fluxos.

Modificados (8):

Arquivo	Alteração	Motivo
<code>src/lib/receber.ts</code>	<code>criarAdiantamento</code> (imediato) agora grava <code>account_id</code> no <code>payments</code>	Adiantamento pago agora é caixa; antes perdia o vínculo.
<code>src/app/(app)/reservas/actions.ts</code>	<code>addPayment</code> grava <code>account_id</code> ; <code>createReservation</code> repassa a conta do adiantamento	Pagamento na reserva e adiantamento de criação com conta.
<code>src/app/(app)/financeiro/actions.ts</code>	<code>addExpense</code> grava <code>account_id</code>	Saída avulsa com conta.
<code>src/app/(app)/financeiro/page.tsx</code>	Select de Conta Corrente no formulário de saídas; conta exibida nas listas de entradas/saídas	UI do novo campo + visibilidade do vínculo.
<code>src/app/(app)/reservas/[id]/page.tsx</code>	Carrega contas; selects de Conta Corrente em Adiantamento, Parcelamento e Pagamentos; conta exibida nos pagamentos	UI dos novos campos na tela da reserva.

Arquivo	Alteração	Motivo
src/app/(app)/reservas/ReservationForm.tsx	Seção de adiantamento ganha select de Conta Corrente (prop <code>contas</code>)	Adiantamento na criação da reserva com conta.
src/app/(app)/reservas/nova/page.tsx	Passa <code>contas</code> ao formulário	Alimentar o novo select.
src/app/(app)/reservas/[id]/editar/page.tsx	Passa <code>contas</code> ao formulário	Idem, na edição.

Nenhuma migration antiga alterada. Nenhum arquivo excluído.

18. Checklist de dados para seu teste local

O banco local já contém os dados de demonstração (clientes "exemplo", reservas LIMA-001..005, a Conta Corrente). Para reproduzir os cenários:

CONTA CORRENTE

- ☒ "Conta Corrente" ativa (criada/existente — Configurações > Contas).

CLIENTES — os 3 de demonstração servem (Joao Ribeiro, Maria Souza, Carlos Andrade). Para cenários novos, use "+ Cliente" com sufixo (**teste**) para não misturar com reais.

RESERVAS (via Reservas > Nova):

- ☐ sem pagamento (saldo = total);
- ☐ com pagamento parcial (registrar pagamento menor que o total);
- ☐ com pagamento integral (pagar o total → saldo 0);
- ☐ com adiantamento **pago agora** (vira caixa com conta);
- ☐ com adiantamento **agendado** (fica a receber; depois "Confirmar" para virar caixa);
- ☐ com parcelamento (2–3 parcelas → conta a receber) e caução preenchida.

CONTAS A RECEBER (Financeiro > A receber):

- ☐ em aberto (vencimento futuro);
- ☐ vencida (vencimento no passado, sem baixa);
- ☐ parcialmente recebida (Receber com valor menor que a parcela);
- ☐ totalmente recebida (Receber o valor cheio).

CONTAS A PAGAR (Compras):

- ☐ em aberto (parcela futura);
- ☐ paga (baixar parcela).

19. Comandos para teste local (verificados no `package.json`)

```
# 1. Executar migrations no banco LOCAL (aplica 0018 se ainda não aplicada)
npm run db:migrate:local

# 2. Verificar o banco (exemplos)
npx wrangler d1 execute limas-locacoes --local --command "SELECT * FROM financial_accounts WHERE"
npx wrangler d1 execute limas-locacoes --local --file scripts/vincular-conta-corrente.sql # v:

# 3. Testes automatizados
npm test

# 4. Iniciar o projeto (porta 3210)
npm run dev
#   → http://localhost:3210 (login local: admin / admin123 – dados de demonstração)

# 5. Fluxos a exercitar na UI
#   Financeiro: /financeiro (abas Resumo / Entradas / Saídas / A receber / A pagar)
#   Reserva: /reservas → abrir uma reserva → Pagamentos / Adiantamento / Parcelamento
#   Dashboard: /dashboard (Financeiro do mês)
```

20. Confirmação final de Git

- Branch criada e em uso: `feature/financeiro-conta-corrente` ✓
- Commit: **NÃO REALIZADO** ✓
- Push: **NÃO REALIZADO** ✓
- Deploy: **NÃO REALIZADO** ✓
- Merge na main: **NÃO REALIZADO** ✓

Todas as alterações estão apenas na sua working tree local e no banco D1 local, prontas para a sua validação.